

PROGRAMMING LANGUAGES FOR DATA ANALYSIS

Python, R, SQL, and Rust Ecosystems

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



Outline

- 1 Overview
- 2 Programming Languages
- 3 Language Comparison and Selection



Outline

- 1 Overview
- 2 Programming Languages
- 3 Language Comparison and Selection



Why Programming Languages Matter for Data Analysis

The Data Science Stack

Modern data analysis requires multiple languages optimized for different tasks:

- Data wrangling, exploration, and visualization

- Statistical computing and modeling

- Database queries and data retrieval

- High-performance batch and stream processing

Petabytes/day



Why Programming Languages Matter for Data Analysis

The Data Science Stack

Modern data analysis requires **multiple languages** optimized for different tasks:

- **Data wrangling, exploration, and visualization**
- **Statistical computing and modeling** *ML*
- Database queries and data retrieval
- High-performance batch and stream processing



Why Programming Languages Matter for Data Analysis

The Data Science Stack

Modern data analysis requires **multiple languages** optimized for different tasks:

- Data wrangling, exploration, and visualization
- Statistical computing and modeling
- Database queries and data retrieval
- High-performance batch and stream processing

docx
pdf
tiff

billions } 3-4 millions / PSQL
10 millions / MySQL
snowflake



Why Programming Languages Matter for Data Analysis

The Data Science Stack

Modern data analysis requires **multiple languages** optimized for different tasks:

- **Data wrangling, exploration, and visualization**
- **Statistical computing and modeling**
- **Database queries and data retrieval**
- **High-performance** batch and stream processing

group

real-time



Language Selection Criteria

Performance Factors:

- Execution speed
- Memory efficiency
- Parallelization
- Scalability

Practicality Factors:

• Ecosystem maturity

• Learning curve

• Scalability

• Integration capabilities

Java → heap size
full memory usage

threads
↓
Concurrency
C

No CONDOE
Hack
(PHP)



Language Selection Criteria

Performance Factors:

- Execution speed
- Memory efficiency
- Parallelization
- Scalability

Practicality Factors:

- Ecosystem maturity
- Learning curve
- Community support
- Integration capabilities

Python
↓
C

CPython

main.py
katie
↳ main.pyc



Outline

- 1 Overview
- 2 Programming Languages**
- 3 Language Comparison and Selection



Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths

Inter. → modern



Q6
Speaker

Inter.

Compta

P. ① 2d. Cert. trans. 



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- **Largest ecosystem:** NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- Gentle learning curve: clear syntax, beginner-friendly
- Interactive development: Jupyter notebooks for exploratory analysis
- Production-ready: from prototypes to deployment-grade systems



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- **Largest ecosystem:** NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- **Gentle learning curve:** clear syntax, beginner-friendly
- **Interactive development:** Jupyter notebooks for exploratory analysis
- **Production-ready:** from prototypes to deployment-grade systems



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- **Largest ecosystem:** NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- **Gentle learning curve:** clear syntax, beginner-friendly
- **Interactive development:** Jupyter notebooks for exploratory analysis
- **Production-ready:** from prototypes to deployment-grade systems

biologists, mathematicians,
medical doctors, ...



Python: The Lingua Franca of Data Science

Python

Python is a **high-level, interpreted** general-purpose language with exceptional **ecosystem support** for data analysis, machine learning, and scientific computing. **Readability** and **rapid prototyping** are core strengths.

Strengths:

- **Largest ecosystem**: NumPy, pandas, scikit-learn, TensorFlow, PyTorch
- **Gentle learning curve**: clear syntax, beginner-friendly
- **Interactive development**: Jupyter notebooks for exploratory analysis
- **Production-ready**: from prototypes to deployment-grade systems

ML
Sto.
Maths

fast API
Django
Flask

web
service



Python Core Libraries I

Scientific Computing Stack

- **NumPy** (numpy): n-dimensional arrays, linear algebra, and mathematical functions. Foundation for all scientific Python.

- Pandas (pandas): DataFrames for tabular data manipulation, cleaning, transformation and analysis.

- Matplotlib (matplotlib): low-level plotting library; fine-grained control over visualizations

- Seaborn (seaborn): statistical visualization built on matplotlib; elegant statistical plots with minimal code.

NumPy 1.6

2.0

Pandas

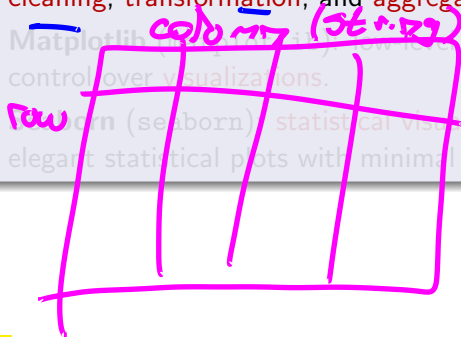
X SQLAlchemy
Pyreppg
Brew Plot



Python Core Libraries I

Scientific Computing Stack

- **NumPy** (numpy): n-dimensional **arrays**, **linear algebra**, and **mathematical functions**. Foundation for all scientific Python.
- **Pandas** (pandas): **DataFrames** for **tabular data manipulation**, **cleaning**, **transformation**, and **aggregation**.
- **Matplotlib** (matplotlib): low-level plotting library; fine-grained control over visualizations.
- **Seaborn** (seaborn): statistical visualization built on Matplotlib; elegant statistical plots with minimal code.



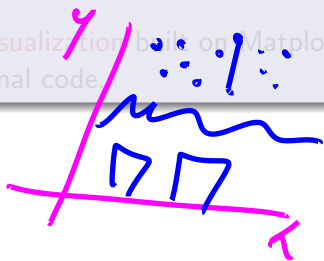
→ Python



Python Core Libraries I

Scientific Computing Stack

- **NumPy** (numpy): n-dimensional **arrays**, **linear algebra**, and **mathematical functions**. Foundation for all scientific Python.
- **Pandas** (pandas): DataFrames for **tabular data manipulation**, **cleaning**, **transformation**, and **aggregation**.
- **Matplotlib** (matplotlib): low-level **plotting** library; fine-grained control over **visualizations**.
- **Seaborn** (seaborn): **statistical visualization** built on Matplotlib; elegant statistical plots with minimal code.



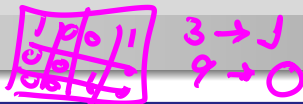
Python Core Libraries I

Scientific Computing Stack

- **NumPy** (numpy): n-dimensional **arrays**, **linear algebra**, and **mathematical functions**. Foundation for all scientific Python.
- **Pandas** (pandas): DataFrames for **tabular data manipulation**, **cleaning**, **transformation**, and **aggregation**.
- **Matplotlib** (matplotlib): low-level **plotting** library; fine-grained control over **visualizations**.
- **Seaborn** (seaborn): **statistical visualization** built on Matplotlib; elegant statistical plots with **minimal code**.



Python Core Libraries II



Statistical Learning

- **SciPy** (scipy): statistical tests, optimization, signal processing, and sparse matrices.
 - **scikit-learn** (sklearn): machine learning — classification, regression, clustering, dimensionality reduction.
 - **statsmodels** (statsmodels): statistical modeling, regression, time series, and hypothesis testing.
- Handwritten pink notes:* An arrow points from SciPy to "empty > 30%". Another arrow points from "empty > 30%" to "statsmodels". A third arrow points from "statsmodels" to "(72mpg)".

Deep Learning & Big Data

- **TensorFlow / PyTorch**: deep learning frameworks
- **PySpark**: distributed processing on clusters



Python Core Libraries II

Statistical Learning

- **SciPy** (scipy): **statistical** tests, optimization, signal processing, and **sparse matrices**.
 - **scikit-learn** (sklearn): **machine learning** — classification, regression, clustering, dimensionality reduction.
 - **Statsmodels** (statsmodels): **statistical modeling**, regression, time series, and hypothesis testing.
- Handwritten note: An arrow points from 'machine learning' to 'classification'.*

Deep Learning & Big Data

- **TensorFlow / PyTorch**: **deep learning** frameworks
- **PySpark**: **distributed processing** on clusters



Python Core Libraries II

Statistical Learning

- **SciPy** (scipy): **statistical** tests, optimization, signal processing, and **sparse matrices**.
- **scikit-learn** (sklearn): **machine learning**— classification, regression, clustering, dimensionality reduction.
- **Statsmodels** (statsmodels): **statistical modeling**, regression, time series, and hypothesis testing.

Deep Learning & Big Data

- **TensorFlow / PyTorch**: **deep learning** frameworks
- **PySpark**: **distributed processing** on clusters



Python Core Libraries II

Statistical Learning

- **SciPy** (scipy): **statistical** tests, optimization, signal processing, and **sparse matrices**.
- **scikit-learn** (sklearn): **machine learning**— classification, regression, clustering, dimensionality reduction.
- **Statsmodels** (statsmodels): **statistical modeling**, regression, time series, and hypothesis testing.

Deep Learning & Big Data

- **TensorFlow / PyTorch**: **deep learning** frameworks
- **PySpark**: **distributed processing** on clusters



↓
Terabytes (Petabytes)

last 10 years

LCMS
billions
neurons



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- Statistical rigor: comprehensive hypothesis testing and model diagnostics
- Visualization excellence: publication-quality plots via ggplot2



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- Visualization excellence: publication-quality plots via ggplot2
- Functional programming: first-class functions, vectorization
- Research community: latest statistical methods often appear in R first
- Reproducible research: R Markdown for literate programming



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via `ggplot2`
- Functional programming: first-class functions, vectorization
- Research community: latest statistical methods often appear in R first
- Reproducible research: R Markdown for iterative programming

paper → IEEE
Scopus



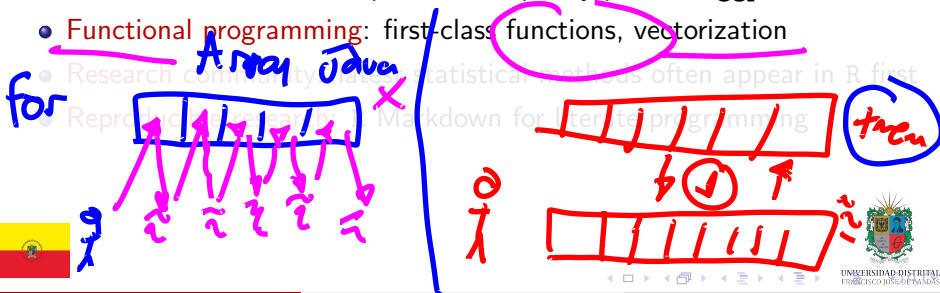
R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via ggplot2
- **Functional programming**: first-class functions, vectorization



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via ggplot2
- **Functional programming**: first-class functions, vectorization
- **Research community**: latest statistical methods often appear in R first
- **Reproducible research**: R Markdown for literate programming



R: The Statistician's Language

R Language

R is a **domain-specific language** for *statistical computing*, data visualization, and graphics. Built by statisticians, for statisticians.

Strengths:

- **Statistical rigor**: comprehensive hypothesis testing and model diagnostics
- **Visualization excellence**: publication-quality plots via ggplot2
- **Functional programming**: first-class functions, vectorization
- **Research community**: latest statistical methods often appear in R first
- **Reproducible research**: **R Markdown** for literate programming



R Core Packages

Data Manipulation:

- **dplyr**: grammar of **data manipulation**
- **tidyr**: **reshaping** and pivoting
- **data.table**: high-performance tables

Statistics & Visualization:

- **ggplot2**: layered graphics
- **psych**: descriptive statistics
- **lme4**: mixed-effects models
- **caret**: ML model training

Tidyverse Philosophy

The **Tidyverse** ecosystem (**dplyr**, **ggplot2**, **tidyr**, etc.) emphasizes **consistent APIs**, **tidy data principles**, and readable pipelines that flow naturally from left to right.



R Core Packages

Data Manipulation:

- dplyr: grammar of **data manipulation**
- tidyr: **reshaping** and pivoting
- data.table:
high-performance tables

Statistics & Visualization:

- ggplot2: layered **graphics**
- psych: descriptive **statistics**
- lme4: mixed-effects models
- caret: **ML** model training

→ **Seaborn**

Tidyverse Philosophy

→ **Pandas**

The **Tidyverse** ecosystem (dplyr, ggplot2, tidyr, etc.) emphasizes **consistent APIs**, **tidy data principles**, and readable pipelines that flow naturally from left to right.



R Core Packages

Data Manipulation:

- dplyr: grammar of **data manipulation**
- tidyr: **reshaping** and pivoting
- data.table: **high-performance** tables

Statistics & Visualization:

- ggplot2: layered **graphics**
- psych: descriptive **statistics**
- lme4: mixed-effects models
- caret: **ML** model training

is it learn

Tidyverse Philosophy

The **Tidyverse** ecosystem (dplyr, ggplot2, tidyr, etc.) emphasizes **consistent APIs**, **tidy data principles**, and readable pipelines that flow naturally from left to right.

Researches



SQL: The Data Query Language

YAML

Structured Query Language (SQL)

SQL is a **declarative language** for querying, filtering, joining, and aggregating data in relational databases. Essential for accessing data at scale.

what, not how
SELECT & FROM &

Core Strengths

Data Warehouse
(relational)



SQL

Reports
Analytics
Prediction
ML



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- Efficient data access: indexes, query optimization, pushdown
- Set operations: JOINS, GROUP BY, window functions
- ACID transactions: data consistency guarantees
- Vendor compatibility: PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake
- Scalability: handles terabytes with distributed query execution

oracle



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access:** indexes, query optimization, pushdown
- **Set operations:** JOINS, GROUP BY, batches
- **ACID transactions:** data consistency guarantees
- **Vendor compatibility:** PostgreSQL, MySQL, Oracle, BigQuery, Snowflake
- **Scalability:** handles terabytes with distributed query execution



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access:** indexes, query optimization, pushdown
- **Set operations:** JOINS, GROUP BY, window functions
- **ACID transactions:** data consistency guarantees
- **Vendor compatibility:** PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake
- **Scalability:** handles terabytes with distributed query execution



SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access**: indexes, query optimization, pushdown
- **Set operations**: JOINS, GROUP BY, window functions
- **ACID transactions**: data consistency guarantees
- **Vendor compatibility**: PostgreSQL, MySQL, SQL Server, BigQuery,

Snowflake

3-4 millions GCP

20 millions ↑



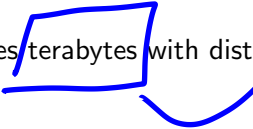
SQL: The Data Query Language

Structured Query Language (SQL)

SQL is a **declarative language** for *querying, filtering, joining, and aggregating* data in relational databases. Essential for accessing data at scale.

Core Strengths:

- **Efficient data access**: indexes, query optimization, pushdown
- **Set operations**: JOINS, GROUP BY, window functions
- **ACID transactions**: data consistency guarantees
- **Vendor compatibility**: PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake
- **Scalability**: handles terabytes with distributed query execution

 Spark-Iceberg



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

SNOWFLAKE

st.

source



```
INSERT T.2
SELECT
  TJ.name,
  AVG(TJ.grats)
FROM
  students;
```



T.2

kw

10000.000
~ 1 seg



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks and lag/lead operations

descriptive
analytics

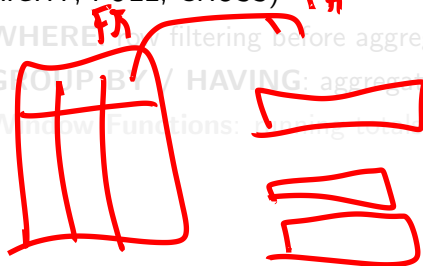


SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS) **PK**
- **WHERE**: filtering before aggregation **FK**
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: ranking, totals, ranks, and lag/lead operations



SQL Operations



Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks, and lag/lead operations

Regex

* → any repetitions
 + → at least one
 . / , → any



SQL Operations

A [2, 4, 7]
B [3, 5]
C [6]

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks and lag/lead operations

count() / sum (goals)

C1	C2
A	2
B	3
A	4
B	5
C	6
A	7

SELECT C1, AVG(C2)
FROM T
GROUP BY C1;
UNIQUE



SQL Operations

Core SQL Clauses

SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY

- **SELECT**: choose columns and computed expressions
- **FROM / JOIN**: source tables and relational joins (INNER, LEFT, RIGHT, FULL, CROSS)
- **WHERE**: row filtering before aggregation
- **GROUP BY / HAVING**: aggregation and post-aggregation filtering
- **Window Functions**: running totals, ranks, and lag/lead operations



Connecting Python/R to Databases

Python Connectors:

- psycopg2: PostgreSQL
- pymysql: MySQL
- SQLAlchemy: ORM and unified interface
- pandas.read_SQL(): direct to DataFrame

psycopg2 - binary

Object
relational
Mapping

No
SQL

Best Practice

Use SQL for heavy lifting—filtering, joining, aggregating large datasets close to the database. Bring curated results into Python/R for analysis & visualization.



list
of
rows

list
of
objects

Class TJ {
attr C1
attr C2



Connecting Python/R to Databases

Python Connectors:

- psycopg2: PostgreSQL
- pymysql: MySQL
- SQLAlchemy: ORM and unified interface
- `pandas.read_SQL()`: direct to DataFrame

R Connectors:

- DBI: database interface
- dplyr::tbl(): lazy SQL queries
- dbplyr: translate dplyr to SQL
- RPostgres, RMySQL *SQL*

Best Practice

Use SQL for heavy lifting—filtering, joining, aggregating **large datasets** close to the database. Bring **curated results** into Python/R for *analysis & visualization*.



Connecting Python/R to Databases

Python Connectors:

- psycopg2: PostgreSQL
- pymysql: MySQL
- SQLAlchemy: ORM and unified interface
- pandas.read_SQL(): direct to DataFrame

R Connectors:

- DBI: database interface
- dplyr::tbl(): lazy SQL queries
- dbplyr: translate dplyr to SQL
- RPostgres, RMySQL

Best Practice

Use SQL for heavy lifting—filtering, joining, aggregating large datasets close to the database. Bring curated results into Python/R for *analysis & visualization*.

Pre-processing



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines **C-level performance** with **memory safety** via an ownership model. No **garbage collection**, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

→ Mozilla (Firefox)

Python →

syntax

to defeat JS.



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- **Blazing speed**, compile-time optimizations, zero-cost abstractions
- Safe parallelism: data races prevented at compile time
- No GC pauses: predictable latency for real-time streaming
- Python bindings: Polars, PyArrow use Rust cores



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- **Blazing speed**: compile-time optimizations, zero-cost abstractions
- **Safe parallelism**: data races prevented at compile time
- **No GC pauses**: predictable latency for real-time streaming
- **Python bindings**: Polars, PyArrow use Rust cores



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- **Blazing speed**: compile-time optimizations, zero-cost abstractions
- **Safe parallelism**: data races prevented at compile time
- **No GC pauses** predictable latency for real-time streaming
- **Python bindings**: Polars, PyArrow use Rust cores



Rust: Systems Programming for Data

Rust Language

Rust is a **systems programming language** that combines C-level performance with memory safety via an ownership model. No garbage collection, no null pointer errors, fearless concurrency.

Why Rust for Data Analysis?

- **Blazing speed**: compile-time optimizations, zero-cost abstractions
- **Safe parallelism**: data races prevented at compile time
- **No GC pauses**: predictable latency for real-time streaming
- **Python bindings**: Polars, PyArrow use Rust cores

popular



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): ultra-fast DataFrame library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- Apache Arrow (arrow2): columnar in-memory format; zero-copy data sharing between systems.
- ndarray (ndarray): N-dimensional arrays (NumPy equivalent).
- Rayon (rayon): data-parallel iterators with work-stealing.

Production Use Cases

- Building high-throughput ETL pipelines
- Real-time stream processing for IoT and finance
- Compiling Python libraries that must be blazingly fast



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work-stealing.

~ relational database

~ CSV
Excel

Production Use Cases

- Building high-throughput ETL pipelines
- Real-time stream processing for IoT and financial data
- Compiling Python libraries that must be extremely fast



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work stealing

Production Use Cases

- Building high-throughput **ETL pipelines**
- **Real-time stream processing** for IoT and finance
- **Compiling Python libraries** that must be blazingly fast



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work-stealing.

Production Use Cases

- Building high-throughput **ETL pipelines**
- **Real-time stream processing** for IoT and finance
- **Compiling Python libraries** that must be blazingly fast



Rust Ecosystem for Data

Key Data Libraries

- **Polars** (polars): **ultra-fast DataFrame** library with Python API. Lazy evaluation, query optimization, Apache Arrow backend.
- **Apache Arrow** (arrow2): **columnar in-memory format**; zero-copy data sharing between systems.
- **ndarray** (ndarray): N-dimensional **arrays** (NumPy equivalent).
- **Rayon** (rayon): **data-parallel iterators** with work-stealing.

Production Use Cases

- Building high-throughput **ETL pipelines** *→ each/row*
- **Real-time stream processing** for IoT and finance
- **Compiling Python libraries** that must be **blazingly fast**



Outline

- 1 Overview
- 2 Programming Languages
- 3 Language Comparison and Selection



Language Comparison Matrix

3

2

2

Criterion	Python	R	SQL	Rust
Ease of Learning	High	Medium	High	Low
Ecosystem Size	Largest	Very Large	N/A	Growing
Performance	Medium	Medium	Excellent	Excellent
Data Manipulation	Excellent	Excellent	N/A	Good
Visualization	Good	Excellent	N/A	Basic
Statistical Tests	Very Good	Excellent	N/A	Limited
Parallelization	Good	Limited	Built-in	Excellent
Production Deployment	Excellent	Good	Standard	Very Good
Interactive Development	Excellent	Very Good	Limited	Poor
ML / DL Support	Dominant	Good	None	Growing
Community Size	Largest	Large	Mature	Growing



When to Use Each Language

Python

Choose Python when you need rapid development, ML/DL frameworks, interactive notebooks, or production deployment with diverse packages.

R

Choose R when you need statistical rigor, publication-quality plots, or work in academia/biostatistics where it is the standard.

SQL

Choose SQL for querying databases, large-scale aggregations, and pushing computation close to data. Always pair with Python/R for analysis.

Rust

Choose Rust when you need maximum performance, safe concurrency, or building mission-critical infrastructure that processes petabytes daily.

cloud computing

When to Use Each Language

Python

Choose Python when you need **rapid development**, **ML/DL** frameworks, interactive notebooks, or **production deployment** with diverse packages.

R

Choose R when you need **statistical rigor**, **publication-quality plots**, or work in **academia/biostatistics** where R is the standard.

SQL

Choose SQL for **querying** databases, large-scale **aggregations**, and pushing computation close to data. Always pair with Python/R for **analysis**.

Rust

Choose Rust when you need **maximum performance**, safe **concurrency**, or building mission-critical infrastructure that processes **petabytes daily**.

When to Use Each Language

Python

Choose Python when you need **rapid development**, **ML/DL** frameworks, interactive notebooks, or **production deployment** with diverse packages.

R

Choose R when you need **statistical rigor**, **publication-quality plots**, or work in **academia/biostatistics** where R is the standard.

SQL

Choose SQL for **querying** databases, large-scale **aggregations**, and pushing computation close to data. **Always pair with Python/R for analysis.**

Rust

Choose Rust when you need **maximum performance**, **safe concurrency**, or building mission-critical infrastructure that processes **petabytes daily**.

When to Use Each Language

Python

Choose Python when you need **rapid development**, **ML/DL** frameworks, interactive notebooks, or **production deployment** with diverse packages.

R

Choose R when you need **statistical rigor**, **publication-quality plots**, or work in **academia/biostatistics** where R is the standard.

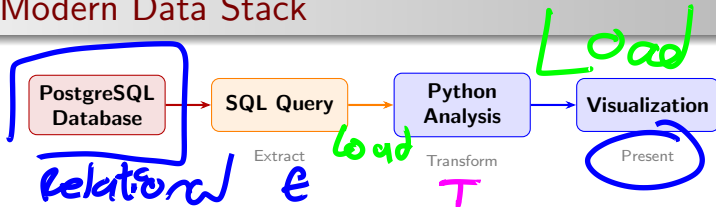
SQL

Choose SQL for **querying** databases, large-scale **aggregations**, and pushing computation close to data. Always pair with Python/R for **analysis**.

Rust

Choose Rust when you need **maximum performance**, safe **concurrency**, or building mission-critical infrastructure that processes **petabytes daily**.

The Modern Data Stack



- Layer 1 (SQL): Query database, filter, aggregate at source
- Layer 2 (Python/R): Load curated data, explore, model, validate
- Layer 3 (Visualization): Create dashboards, reports, and presentations

GCL → 1st analysis
 2nd save
 GCS → 1st save
 2nd analysis?



Example: Multi-Language Pipeline

Scenario: Analyze Sales Data

1 **SQL** Extract monthly sales by region from data warehouse

- Filter to last 12 months
- Group by region, product category
- Calculate totals and growth rates

GROUP BY

2 **Python**: Load results, perform statistical analysis

- Detect outliers and anomalies
- Fit predictive model (scikit-learn)
- Calculate correlations

SELECT region, sum(sales)

FROM sales

3 **R** (optional): Create publication-quality visualization

- ggplot2 for advanced graphics
- Statistical annotations

JOIN ON sales.region = region.id

WHERE date >= ("2027")

GROUP BY region, category



Example: Multi-Language Pipeline

Scenario: Analyze Sales Data

- ① **SQL:** Extract monthly sales by region from data warehouse
 - Filter to last 12 months
 - Group by region, product category
 - Calculate totals and growth rates
- ② **Python:** Load results, perform statistical analysis
 - Detect outliers and anomalies
 - Fit predictive model (scikit-learn)
 - Calculate correlations
- ③ **R (optional):** Create publication-quality visualization
 - ggplot2 for advanced graphics
 - Statistical annotations

Extract



Example: Multi-Language Pipeline

Scenario: Analyze Sales Data

- ① **SQL**: Extract monthly sales by region from data warehouse
 - Filter to last 12 months
 - Group by region, product category
 - Calculate totals and growth rates
- ② **Python**: Load results, perform statistical analysis
 - Detect outliers and anomalies
 - Fit predictive model (scikit-learn)
 - Calculate correlations
- ③ **R (optional)**: Create publication-quality visualization
 - ggplot2 for advanced graphics
 - Statistical annotations

API



Best Practices for Multi-Language Work

- **Separation of concerns:** Each language handles what it does best
- **Data formats:** Use Parquet or CSV as interchange format
- **Version control:** Track code and data pipelines in Git
- **Reproducibility:** Document dependencies and environment setup
- **Profiling:** Measure performance, identify bottlenecks
- **Testing:** Validate transformations at each stage



Outline

- 1 Overview
- 2 Programming Languages
- 3 Language Comparison and Selection



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/data-analysis-programming>

